hashlock.

2024

Security Audit

Natix Network (DePIN)

July 2024 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Natix Network team partnered with Hashlock to conduct a security audit of their Staking and Voting programs. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

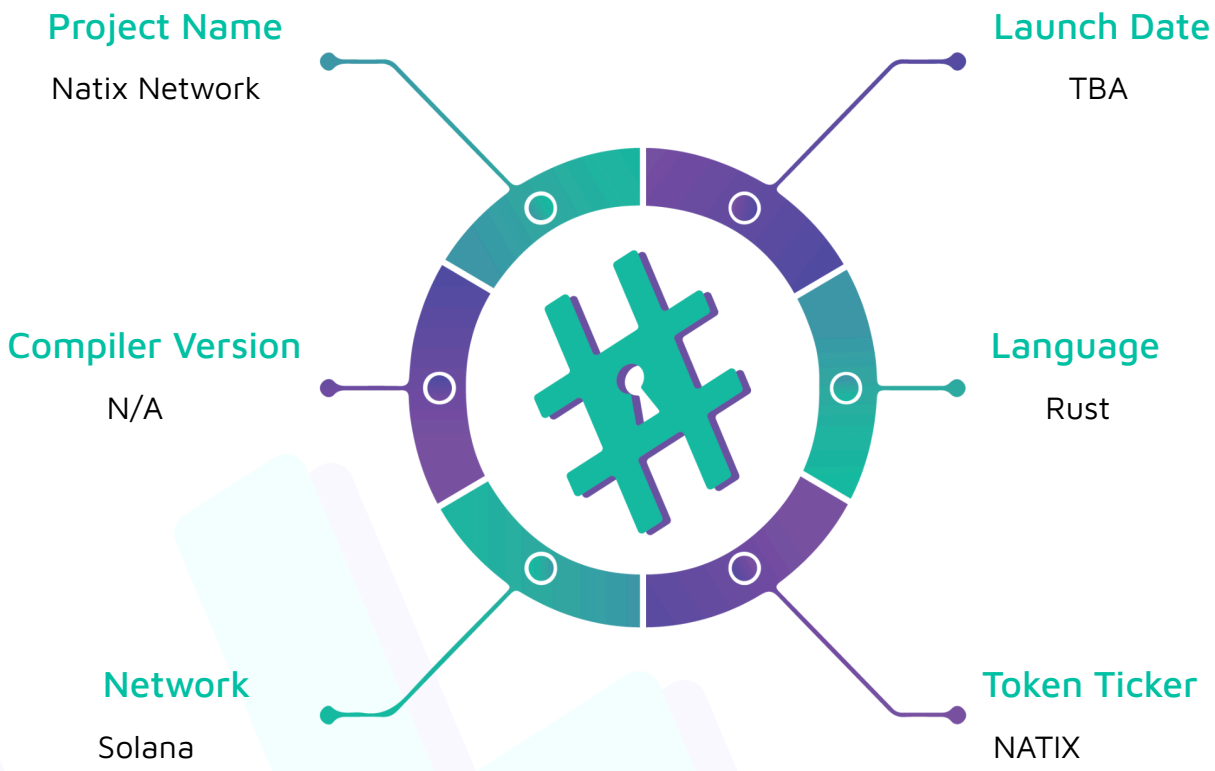
Project Context

Natix Network is a cutting-edge blockchain project dedicated to enhancing the interoperability and scalability of decentralized applications (dApps) and blockchain networks. With a focus on solving the challenges of current blockchain ecosystems, Natix Network leverages advanced protocols and infrastructure to facilitate seamless cross-chain communication and transactions.

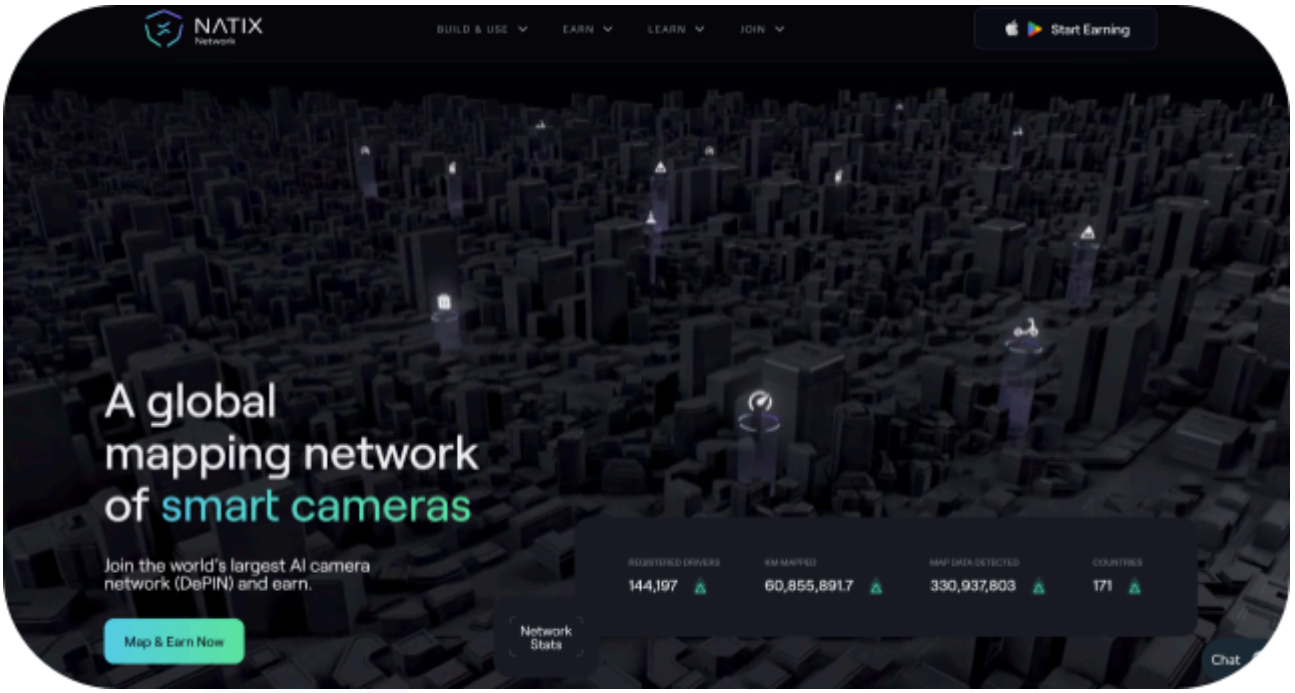
At its core, Natix Network aims to empower developers and users by providing a unified platform that supports decentralized finance (DeFi) applications, gaming ecosystems, and other decentralized applications. By enabling secure and efficient interactions between different blockchain networks, Natix Network fosters innovation and growth within the broader blockchain community.

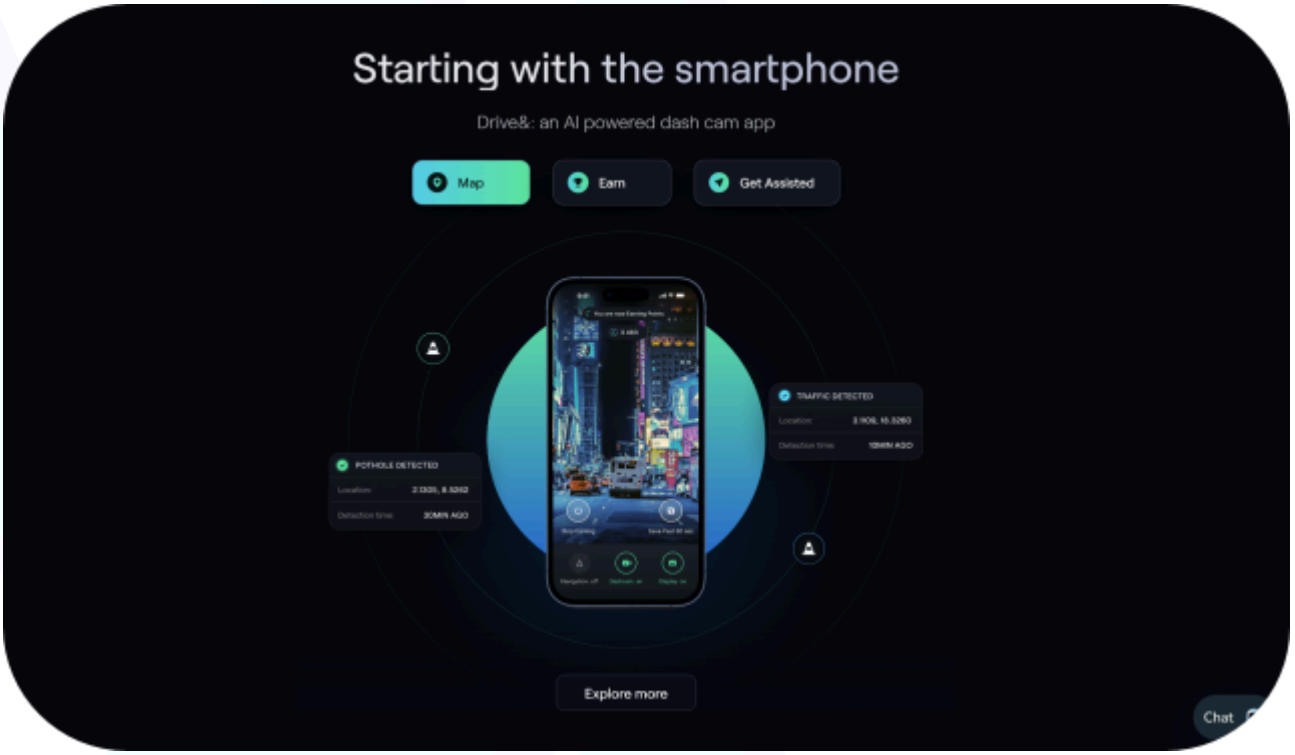
Project Name: Natix Network
Compiler Version: N/A
Website: https://www.natix.network/
Logo:

Visualised Context:



Project Visuals:





Audit scope

We at Hashlock audited the solidity code within the Natix Network project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Natix Network Programs**
Platform	**Solana / Rust**
Audit Date	**July, 2024**
Program 1	stake_V1
GitHub Commit Hash	2bf22d4622ecf96a88cf3950d59e0cc706f5312f

Security Rating

After Hashlock's Audit, we found the programs to be **"Secure"**. The programs all follow simple logic, with correct and detailed ordering. They rely on a series of SPL programs.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

1 High-severity vulnerabilities

2 Medium-severity vulnerabilities

5 Low-severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
Staking v1 Allows users to: - Stake tokens - Unstake tokens - Unstake tokens immediately - Withdraw - Cancel Allows admins to: - Pause / Unpause - Configure	**The program achieves this functionality.**

Code Quality

This audit scope involves the smart contracts for the Natix Network project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Natix Network projects smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help us understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] StakeProgram - Tokens can get locked up indefinitely

Description

Tokens can be staked from any Natix token account a user owns (which can be many). However, only tokens in one specific account per user (the ATA) can be unstaked and therefore retrieved.

Vulnerability Details

```
control_user_solana_address(user_account, authority_account)?;
let address = get_associated_token_address(user_solana_account.key,
&get_natix_token_mint());

if user_account.key != &address {
     return Err(ProgramError::InvalidAccountOwner);
}
```

Impact

The unstake and withdraw functions require the token account to be the ATA (associated token account). Staking can be done from any token account the user has. This means that all funds staked from any user token account that is not the ATA will be stuck indefinitely.

Recommendation

Either allow staking from the ATA only or also support all accounts for unstaking, withdrawal, etc.

Status

Resolved

Medium

[M-01] Staking Program - Risk of overflow & underflows

Description

Certain maths operations are at risk of overflow or underflow.

Vulnerability Details

```
let sum_stakes = amount.unwrap_or(0) + give_stakes(staking_pool,
address).iter().map(|staker| staker.amount).sum::<u64>();
```

This is partially user input and could be made to overflow.

```
amount += (unstaker.amount + reward - penalty).max(0);
forfeit_amount += (penalty - reward).max(0);
```

These operations can underflow if there are configuration mistakes.

Impact

Incorrect configurations could lead to broken accounting calculations due to overflows or underflows.

Recommendation

Use checked math for calculations involving user inputs or configurable variables.

Status

Resolved

[M-02] Staking Program - Absence of integration tests

Description

Integration tests are crucial to ensure the program works correctly. While some unit tests exist, there are no tests for the overall functionality, and there's a lack of code documentation.

Vulnerability Details

Lack of complete integration tests.

Impact

Without integration tests, bugs may go undetected, making future modifications risky. Security concerns are also harder to test.

Recommendation

Implement comprehensive integration tests that cover all parts of the program, including scenarios where things should not work. Use random numbers to test reward calculations to identify potential rounding issues.

Note:

The Natix team will present the integration tests at a later date.

Status

Acknowledged

Low

[L-01] Staking Program - Weak account checks

Description

Although secure due to Solana's token program guarantees, some account checks should be added to improve code maintainability.

Vulnerability details

```
pub fn control_user_account(user_account: &AccountInfo) -> ProgramResult {
     let user_token_account = TokenAccount::unpack(&user_account.data.borrow())?;

     msg!("Checking user token account mint address {:?}", user_token_account.mint);

     if user_token_account.mint != get_natix_token_mint() {
     return Err(ProgramError::IncorrectProgramId);
     }

     Ok(())
}
```

The `transfer_token` function does not verify if the source and receiver accounts are the same. There are other similar checks that could be improved for better maintainability.

Impact

Future changes could break these checks, leading to serious issues.

Recommendation

Properly verify which programs own which accounts.

Status

Resolved

[L-02] Voting & Staking Program - Voting is optional, voters can vote for all options as many times as they please

Description

The vote function allows voters to vote on any question as many times as they please. They can also vote for all answers a question has. Also, there is no need to vote at all, as the instruction on the staking contract can be called directly. The staking contract does not check if the vote program was first called within the same transaction. This makes voting purely optional.

Finally, the `stake_program_id` is never checked to be the stake program. As this is user input, any program could be called here.

Vulnerability details

```
pub fn vote<'a>(mut data: Data, user_account: &'a AccountInfo<'a>, ..., questionId:
String, answerId: String) -> ProgramResult {
      msg!("Voting from user account {:?} ", user_account.key);
      control_provisioned(&data)?;
      control_user_solana_address(user_account, user_solana_account)?;


      let clock = Clock::get()?;

      let questions: Vec<&mut Question> = data.questions.iter_mut().filter(|q|
q.expired_at > clock.unix_timestamp && q.id == questionId).collect();

      if (questions.len() > 0) {
      for q in questions {
            if q.answers.contains(&answerId) {
            let index = q.answers.iter().position(|a| *a == answerId).unwrap();
            q.stats[index] += 1;
            }
      }
      }
```

Impact

The impact is unclear as there is no documentation indicating the expected functionality.

Recommendation

Consider what the requirements are and implement the functionality accordingly. Also, update the in-code documentation to reflect what the voting program does. The

documentation is mostly copied from the staking program and describes what that does instead.

Status

Resolved

[L-03] Staking Program - Code Optimizations

Description

The code can sometimes be simplified

Vulnerability details

```
match pool {
      Ok(p) => p,
      Err(e) => Data {
      questions: vec![],
      provisioned: false,
      },
}
```

Such patterns can be done with unwrap_or_else.

```
/// Reads the program_account and returns StakePool struct
```

Some of the very rare comments are copy & pasted or don't describe what is happening.

```
match Data::try_from_slice(instruction_data) {
      Ok(data) => {
      let instruction = data.instruction;
```

Use unwrap_or_else, there is no need for explicit return statements here.

```
amount += (unstaker.amount + reward - penalty).max(0);
forfeit_amount += (penalty - reward).max(0);
amount += (unstaker.amount + reward).max(0)
if amount <= 0 {
```

Some of the code uses max(0) on unsigned integers. This does nothing as an unsigned integer can't be less than 0.

Impact

An unnecessarily complicated code base decreases the maintainability.

Recommendation

Make sure to write easy-to-follow code. Use *cargo fmt* to format the source code.

Status

Resolved

[L-04] Staking Program - Deposited liquidity is split

Description

Tokens that are staked have to be deposited into a token account owned by a PDA of the program. While this allows for some flexibility, it also implies that liquidity is split.

Vulnerability details

Not a security concern.

Impact

There could, in theory, be thousands of token accounts holding a few tokens each. This may make it impossible for a large holder to withdraw funds until they are combined into one token account with a large enough balance.

Recommendation

Consider the pros & cons of each strategy and implement this according to requirements.

Status

Acknowledged

[L-05] Staking Program - Duplicate stakes can happen

Description

It is possible to create two stakes within the same block - that means the data of the item in the vector is identical which qualifies it as the same item.

Vulnerability details

```
stakers.retain(|staker| !valid_unstakers.contains(&staker));
```

This function may remove more than one item.

Impact

This code removes items based on all values of a struct being the same. This can be the case if the same amount was staked twice in the same block. However, this means they will both be removed at the same time therefore this works alright.

Recommendation

This is a likely source of errors in future updates. Make sure to at least document this properly if not prevent it from happening in the first place.

Status

Acknowledged

Centralisation

The Natix Network project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlocks analysis, the Natix Network project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au


#Hashlock.